

OptiShift Backend: Final Delivery Report

1. Tech Stack & Foundation

- **Framework:** FastAPI (Async REST API) served via Uvicorn.
- **Data Validation:** Pydantic v2 for strict typing of all requests, responses, and internal models.
- **Optimization Solver:** PuLP utilizing the open-source CBC mathematical solver.
- **Architecture:** Clean 3-tier architecture: API Routes -> Services -> Optimizer/Data.

2. The Core Data Models

Fully typed schemas defining the business logic:

- **Employee:** Includes hourly rate, skills, max weekly hours, and granular day-by-day availability/preferences.
- **Shift:** Defines start/end times, required staff counts, and required skills.
- **Task:** Defines criticality (Critical/High/Medium/Low), priority, owner, and whether replacements are allowed.
- **Leave:** Tracks start/end dates, leave type, and status (Pending/Approved/Rejected).
- **LeaveImpact & ScheduleDiff:** Complex models handling the rippling effects of schedule changes.

3. The MILP Optimization Engine (milp_engine.py)

The heart of the automated scheduling system. It evaluates hundreds of combinations using Mixed-Integer Linear Programming.

- **Hard Constraints (Must be met):**
 - No scheduling employees on days they are unavailable.
 - No scheduling employees who are on approved leave.
 - Maximum 1 shift per employee per day.
 - Strict adherence to max_hours_per_week limits.
 - Exact required_staff counts met for every shift.
 - Strict skill-matching (e.g., must have food_prep and barista to work the afternoon shift).
- **Objective Function (Minimization):**
 - Cost: Prioritizes cheaper labor combinations.
 - Overtime: Penalizes hours extending beyond standard thresholds.
 - Preferences: Penalizes assigning staff to shifts they don't prefer.
 - Fairness: Actively minimizes the gap in total hours between different eligible employees so work is distributed evenly.
- **Day-Aware Diagnostics:** If a schedule is mathematically impossible, it pinpoints the exact day, shift, and skill shortage causing the failure instead of throwing a generic error.

4. The Task Impact & Risk Engine (impact_service.py)

The main differentiator of the app. It analyzes the consequences of a leave request before it is approved.

- **Inseparable Risk + Context:** Adheres strictly to the rule that the backend never returns a generic "HIGH" risk. It returns the risk level explicitly tied to the exact affected_tasks, owner, and replacements.
- **4-Tier Risk Classification:**
 - **CRITICAL:** An affected task is owner-only (replacement_allowed = False) OR no qualified replacement exists in the company.
 - **HIGH:** A critical or high-priority task is affected, but a replacement candidate is available.
 - **MEDIUM:** A standard task requires reassignment, or a shift falls understaffed (but replacements exist).
 - **LOW:** Routine tasks with multiple available candidates.
- **Replacement Candidate Search:** Automatically queries all employees to find who has the matching skills, is available on that day, and has enough room in their max_hours budget to take over the work.

5. Re-optimization & Metrics (schedule_service.py & metrics.py)

- **Before vs. After Diff:** When a manager clicks "Approve", the MILP re-runs and generates a strict JSON diff showing exactly who was removed (removed_assignments) and who was added (added_assignments).
- **Business Metrics:** Quantifies the schedule before and after the leave, calculating: Total Labor Cost, Coverage %, Overtime Hours, Fairness Score (0-100), and Total Savings compared to a baseline.

6. The API Surface (api/)

Fully documented via auto-generated Swagger UI (<http://localhost:8000/docs>).

- GET/POST /api/v1/employees, /shifts, /tasks
- POST /api/v1/optimize: Runs the initial schedule generation.
- POST /api/v1/leave: Submits a leave request.
- GET /api/v1/leave/{id}/impact: Generates the massive Impact Engine report.
- POST /api/v1/leave/{id}/approve & /reject: Manager actions.
- POST /api/v1/reoptimize: Triggers the Before vs. After diff generation.

7. Demo Dataset (demo_data.py)

- **UrbanBrew Cafe, Mumbai:** Fully seeded in-memory database with 8 employees, 3 daily shifts, and a variety of skills.
- **The "Perfect" Demo Scenario:** Programmed specifically around "Priya Sharma". When her Friday leave is requested, it perfectly triggers a multi-level risk report (A CRITICAL owner-only Client Presentation + a MEDIUM shift reassignment), showcasing the full power of the Risk Engine.